
CSE Web Data Collection

Document for 25ada141 - Salaama

Your part: OCR, saving the files, and the charts

Everything behind your part of the project - the ideas, the reasons, and the code - explained step by step, from the beginning.

DA 2009 - Data Collection Methods II

Assignment 2 (Group Project)

University of Colombo

25ada072 - Pasindu | 25ada073 - Sasini | 25ada141 - Salaama | 24ada076 - Nuhan

Contents

How to use this document	3
The project in simple words	4
How the web works	6
The ideas behind your part	9
Reading Python	15
Your file: <code>src/ocr_extractor.py</code>	21
Your file: <code>src/storage.py</code>	26
Your file: <code>src/analysis.py</code>	29
How your part fits into the whole project	32
Ethics - what every member must know	33
Showing your part in the viva	34
Practice questions	36
Word list	43

How to use this document

This document explains your part of the project from the very beginning. It does not assume you have seen any of it before. Each chapter builds on the one before it, so read it in order the first time.

Your files are:

- `src/ocr_extractor.py`
- `src/storage.py`
- `src/analysis.py`

You will see four kinds of coloured box:

Think of it like this. a comparison with something from everyday life, to make a new idea easier to picture.

Remember: the one thing to take away from a section. If you only read the green boxes, you will still know the main points.

Be careful: a trap, or something an examiner might test you on.

Try it yourself. something to type on the computer, and what should happen.

Grey boxes in a typewriter font are **code** - real lines copied from our project files, not made-up examples.

A plan for the week before the viva

Day	What to do
1	Read chapters 1 to 3. They explain the project and the ideas behind your part.
2	Read the chapter on reading Python, with your own files open beside it.
3 and 4	Read the chapters on your files, one at a time. Run each file as the 'Try it yourself' boxes show.
5	Read how your part fits into the whole project, and the ethics chapter.
6	Practise the viva chapter out loud - say your 30-second explanation until it feels natural.
7	Go through the practice questions at the end. Cover the answer and try to say it first.

The project in simple words

What a stock exchange is

A company can split its ownership into many small pieces called **shares**. If you buy one share of a bank, you own a tiny piece of that bank.

A **stock exchange** is the marketplace where people buy and sell those shares. The price of a share goes up when many people want to buy it, and down when many people want to sell it. The **Colombo Stock Exchange** - CSE for short - is Sri Lanka's stock exchange, and its website is www.cse.lk.

Think of it like this. a stock exchange is like a big weekly fair where, instead of vegetables, people trade small pieces of companies. The website is the notice board at the entrance that shows today's prices.

What we were asked to do

Our group was given the CSE website and asked to collect data from it with a computer program, using the methods taught in the course, and to do it **ethically** - politely, legally, and without causing the website any trouble.

What we collect

What	What it means	Example
Share prices	What every company traded at today	Sampath Bank: Rs 141.00
Company details	Facts about one company	Its total value, its highest and lowest price this year
Announcements	Notices companies publish, often as PDF files	'A dividend will be paid', 'A meeting will be held'

After collecting, we tidy the data, check it makes sense, draw charts, and save it as files that other people can open.

Who did what

Member	Part of the project
24ada076 - Nuhan	Ethical scraping and robots.txt, BeautifulSoup, and reading PDFs
25ada072 - Pasindu	Getting the data from the site, and Selenium
25ada073 - Sasini	Crawling for documents, and cleaning the data
25ada141 - Salaama (this is you)	OCR, saving the files, and the charts

The app itself (app.py) was put together by all four of us. Each tab in the app is labelled with the member whose part it shows.

The one surprise that shaped the whole project

The first thing we tried was the ordinary method from the course: download the web page and pick the prices out of it. It found almost nothing - about 24 letters of text, and no tables at all.

The reason is that the CSE website is built with **JavaScript**. The page arrives nearly empty, and a small program inside your browser then fetches the prices and fills them in. A plain download never runs that program, so it never sees the prices.

So we opened the site in a browser, pressed **F12**, and watched the **Network** tab, which lists everything the page asks for. It showed the page fetching its prices from addresses such as `https://www.cse.lk/api/tradeSummary`. We read those addresses directly instead, and one request gives us every company at once.

Remember: the CSE page is built by JavaScript, so we read the data from the same addresses the page itself uses. That one decision explains why the project is built the way it is.

How the web works

Before looking at any code, it helps to understand what actually happens when a computer 'visits' a website. There are only a few ideas, and every part of our project uses them.

Requests and responses

When you open a web page, your computer sends a message to another computer, called a **server**, asking for that page. This message is a **request**. The server sends something back - the page, a file, or some data. That is the **response**.

Think of it like this. writing a letter and getting a reply. You write to an address and ask for something. A little later, a reply arrives in your letterbox. Every single thing a website shows you - each picture, each price - arrives this way, one letter and one reply at a time.

Addresses - URLs

Every page and file on the web has an address, called a **URL**. For example:

```
https://www.cse.lk/api/tradeSummary
```

Part	What it means
https://	How to talk to the server. The 's' means the conversation is locked so nobody can read it on the way.
www.cse.lk	Which server - the CSE website.
/api/tradeSummary	Which thing on that server we want. This part is called the path .

GET and POST - two ways of asking

There are two common kinds of request. A **GET** request simply asks for something: 'please give me this page'. A **POST** request sends some information along with the question, like filling in a form before handing it over.

Most of the CSE data addresses only answer **POST** requests - if you send a GET, they reply with an error. That is their rule, not ours, and we found it by trying both.

Status codes - the result stamped on every reply

Every response comes with a three-digit number that says how it went. These were taught in Week 2:

Code	Meaning	In everyday words
200	OK	Here is what you asked for.
301	Moved Permanently	That has moved - look over there.
403	Forbidden	You are not allowed to have this.
404	Not Found	There is no such page.

Code	Meaning	In everyday words
500	Internal Server Error	Something broke on our side.

Headers - the envelope

Along with every request travel a few lines of extra information called **headers**. One of them, the **User-Agent**, says what program is asking. A browser sends something like 'Chrome on Windows'. Our project sends its own name, the university, the course, and an email address.

Think of it like this. the User-Agent is like writing your name and address on the back of the envelope. The receiver knows who wrote, and can write back if there is a problem.

HTML - what a web page is made of

A web page is a text file written in **HTML**. It uses **tags** - words inside angle brackets - to label each part of the page:

```
<html>
  <head>
    <title>Trade Summary</title>
  </head>
  <body>
    <h1>Today's prices</h1>
    <table>
      <tr> <th>Company</th> <th>Price</th> </tr>
      <tr> <td>Sampath Bank</td> <td>141.00</td> </tr>
    </table>
  </body>
</html>
```

Tags usually come in pairs: <title> opens and </title> closes. Whatever sits between them is the content. Tags sit inside other tags, like boxes inside boxes: the table is inside the body, each row <tr> is inside the table, and each cell <td> is inside a row.

JSON - data in a tidy shape

Websites do not only send pages. They also send plain **data**, and the most common format for that is **JSON**. JSON is made of **names and values**, grouped with curly brackets { }, and lists grouped with square brackets []. This is a real piece of what CSE sends us:

```
{
  "symbol": "SAMP.N00000",
  "name": "SAMPATH BANK PLC",
  "price": 141.0,
  "change": 1.0
}
```

Think of it like this. JSON is like a neatly filled-in form. Every box has a label ('name', 'price') and an answer. A program can read it straight away, with no need to search through the decorations of a web page.

Static and dynamic pages

	Static page	Dynamic page
Where the content is	Already written inside the HTML file	Added by JavaScript after the page arrives
Can a plain download see it?	Yes	No
Example	A list of university departments	CSE's live prices

Think of it like this. a static page is a printed menu - everything is already on it. A dynamic page is an empty chalkboard; a waiter writes today's specials on it after you sit down. If you take a photo the moment you walk in, you photograph an empty board.

Try it yourself. open <https://www.cse.lk/equity/trade-summary> in Chrome or Edge. Right-click and choose **View Page Source** - this shows what the server sent, and you will not find the prices in it. Now right-click a price and choose **Inspect** - this shows the page after JavaScript ran, and the prices are there.

The ideas behind your part

Your part has three pieces. **OCR** reads documents that are really pictures. **Saving** writes the data into files other people can open. **Charts** turn a long table into something a person can understand at a glance. OCR has the most ideas behind it, so most of this chapter is about OCR.

Why the project needed OCR

Sasini's crawler downloads the PDF documents CSE publishes, and Nuhan's code reads the text out of them. For most documents that works: a normal PDF has the letters stored inside it, and the computer can copy them straight out. Nuhan's code counts the characters it finds on each page:

Document	Pages	Characters found	Kind
Employee share option schemes	1	896	Text-based
DFCC Bank debentures	1	792	Text-based
De-listing - Associated Motor Finance	1	0	Scanned
Revised format for disclosure ...	4	0, 127, 1,129, 274	Hybrid - page 1 is a scan

The de-listing notice gives **nothing** - not one character. It looks like an ordinary letter on screen, but it is a **scan**: somebody printed it, signed it, and put it through a scanner. The PDF holds a photograph of the page, not the letters themselves.



The de-listing notice. Every word you can see here is part of a photograph.

A page with fewer than **50** characters is treated as a scan. Of the 12 documents saved when this was written, two need OCR: the de-listing notice, and page 1 of the revised-format document.

Think of it like this. a text-based PDF is a typed letter - you can copy any word out of it. A scanned PDF is a photo of that letter - you can see the words, but to the computer it is just dots of colour. OCR is somebody reading the photo out loud and typing what they read.

What OCR is

OCR stands for **Optical Character Recognition**. The Week 8 lecture defines it as converting images of text into machine-readable text. 'Optical' means it works from a picture; 'character recognition' means it works out which letter each shape is.

The lecture gave real uses of OCR:

- Turning paper invoices, receipts and bank statements into digital records.
- Reading car number plates from road and car-park cameras.
- Reading passports at airport immigration counters.
- Pulling policy numbers, names and amounts out of insurance forms.
- Reading printed text aloud for people who cannot see it.

What a picture is, to a computer

A digital picture is a grid of tiny squares called **pixels**. Each pixel is stored as a **number** for how bright it is: **0** is black, **255** is white, and everything in between is a shade of grey. Zoom right into a grey picture of the letter 'n' and this is all the computer sees:

```
255 255 255 255 255 255
255 38 41 190 52 255
255 35 255 255 40 255
255 36 255 255 44 255
255 255 255 255 255 255
```

The small numbers are dark ink; the 255s are white paper. The computer has no idea this is a letter. OCR has to work that out from the pattern of numbers.

A **colour** picture stores **three** numbers for every pixel - how much red, green and blue light is in it. Mixing those three makes every colour. OpenCV stores them in the order blue, green, red, which is why its setting is called `COLOR_BGR2GRAY`.

Think of it like this. a picture is like a huge mosaic made of tiny tiles. Each tile has a number painted on the back saying how dark it is. The computer never sees the picture - only the numbers.

Resolution and dpi

A PDF page has no pixels until we make an image from it. **dpi** (dots per inch) decides how many pixels to use for each inch of paper. More pixels means more detail - and more work. Our de-listing notice is an A4 page, about 8.3 by 11.7 inches:

dpi	Image size	What it is like
72	about 595 x 841 pixels	Screen size. Small print turns to mush.
200	1,653 x 2,337 pixels (about 3.9 million)	Our setting. Clear for printed documents.

dpi	Image size	What it is like
300	about 2,480 x 3,504 pixels	More detail for very small print, but slower.

The 200 dpi size is the real size of the image our code made from the de-listing notice.

The OCR pipeline - four steps

The Week 8 lecture split OCR into four steps, each with its own tool. Your file follows exactly these steps:

Step	What happens	Tool
1. Image acquisition	Get the picture - here, turn the PDF page into an image	pdfplumber, giving a PIL image
2. Preprocessing	Clean the picture up so the letters are easy to read	OpenCV
3. Text recognition	Read the letters	pytesseract and Tesseract
4. Post-processing	Tidy the text into a table and save it	pandas

Think of it like this. photocopying a faded old letter before trying to read it. First you get the letter onto the copier (step 1). You turn the contrast up so the writing is dark and the paper is white (step 2). Then you read it (step 3), and type it up neatly (step 4).

Preprocessing - why clean the picture?

The lecture's list of OCR challenges starts with **poor image quality**, and its answer is to improve the image before reading it. Tesseract, the reading program, works best on plain black letters on a clean white background. A real scan is not like that: the paper is slightly grey, the ink is uneven, and one side of the page can be darker than the other.

Greyscale

Colour tells us nothing about which letter a shape is - an 'a' is an 'a' in blue or in black. Turning the picture grey leaves **one** number per pixel instead of three, so every later step has a third of the numbers to deal with.

Thresholding - black or white, nothing in between

Thresholding makes every pixel either pure black (0) or pure white (255). You pick a cut-off: anything brighter becomes white, anything darker becomes black. The letters get hard, clean edges.

grey pixels:	250	243	128	36	40	201	248
cut-off 150:	255	255	0	0	0	255	255

The problem is choosing **one** cut-off for the whole page. If the left side of a scan is darker, a cut-off that suits the right side turns the left side's paper black.

Adaptive thresholding fixes this. Instead of one cut-off for the whole page, it works out a separate cut-off for every small area, from the pixels around it. A pixel only turns black if it is darker than its own neighbourhood.

Think of it like this. judging whether someone is tall. One rule for everybody - 'over six feet is tall' - fails in a class of children. Adaptive thresholding compares each person with the people standing around them.

Noise, kernels and morphology

Noise means small marks that are not part of the text - specks of dust on the scanner glass, for example. The lecture's OCR example includes an optional **noise removal** step using a method called **morphology**, which changes the shapes in a black-and-white image.

Morphology slides a tiny grid, called a **kernel**, over every pixel of the image. The kernel's size decides how big a mark it can affect. There are two basic moves:

Move	What it does to the white areas
Erosion	Shrinks them - white shapes get thinner
Dilation	Grows them - white shapes get fatter
Opening	Erosion, then dilation. Removes small white spots
Closing	Dilation, then erosion. Fills in small black spots

A scan of a printed page has **black** specks on **white** paper, so closing is the move that would remove them - the white paper grows over the speck, then shrinks back, and the speck stays filled in.

Be careful: the lecture's example uses **opening** with a **1 x 1** kernel. A 1 x 1 kernel looks at only one pixel at a time, so it cannot change anything. We measured it on the de-listing notice: **0 pixels different** before and after. Your code keeps the step so the pipeline matches the lecture, and your file says honestly that it changes nothing. If you are asked, say exactly that - never claim it removes specks.

Why not just switch to closing with a bigger kernel? Because a full stop is also a small black dot. A kernel big enough to remove a speck of dust can also remove full stops, the dot on an 'i', or thin parts of letters. That change would need testing on real pages before it could be trusted.

How the reading step works

Tesseract is the program that actually reads the letters. The lecture notes that it was first created by HP and later developed by Google, and it is free to use. It finds the lines of text on the page, splits them into words, and recognises the letters using patterns it learned from a large amount of example text. For every word it also gives a **confidence** score from 0 to 100 - how sure it is.

pytesseract is not Tesseract. It is a small Python **wrapper**: it sends our image to the Tesseract program, waits for the answer, and hands it back to Python. That is why there are two things to install:

What	How it is installed	Done?
pytesseract (the Python wrapper)	<code>pip install pytesseract</code> - in requirements.txt	Yes
Tesseract (the actual program)	Downloaded and installed separately on Windows, then added to the PATH	Must be done on the laptop used

Think of it like this. pytesseract is a telephone, and Tesseract is the expert on the other end of the line. You can own a perfectly good telephone, but if nobody is on the other end, you get no answer.

Be careful: until Tesseract is installed, the OCR tab shows installation instructions instead of the pictures and the text. Install it on the laptop you will use for the viva, and check it by typing `tesseract --version` in a terminal.

OCR is not perfect

OCR makes mistakes, and the lecture's last best practice is to **verify** the output - check it against the real page by eye. Some mistakes are common because the shapes really are alike:

Easily confused	Why
0 (zero) and O (letter)	Both are round
1, l (small L) and I (capital i)	All are one straight line
rn and m	Two letters close together look like one
5 and S, 8 and B	Similar curves

Logos are especially hard. The CSE logo at the top of every notice separates its letters with thin lines, and an OCR program may read those lines as extra letters, such as an I or a 1:



The CSE logo from the scanned notice. To OCR, the lines between C, S and E look a lot like letters.

That is what the confidence score is for. Your code lists the words Tesseract was **least** sure about, so a person knows exactly which words to check by hand instead of re-reading everything.

Saving data - three kinds of file

Format	What it is	Good for
CSV	Plain text. One row per line, commas between the columns	Opens in anything - Excel, pandas, even Notepad
Excel (.xlsx)	A spreadsheet file, which can hold several sheets	People who want to sort and filter by hand
JSON	Names and values, like the data CSE sends us	Other programs, and keeping extra details beside the data

A CSV file of our share prices starts like this - it really is just text:

```
symbol,price,change
SAMP.N0000,136.75,-0.25
AEL.N0000,24.5,0.3
```

Crediting the source - inside the file

The Week 6 lecture lists crediting the data source as part of ethical scraping. A spreadsheet gets copied and emailed on, and a note in a separate document gets lost along the way. So every file we save carries the credit **inside itself**: where the data came from, when it was collected, and that it was for academic use.

Think of it like this. the label sewn into a piece of clothing. However many times it is passed on, the label still says where it was made.

Text encoding

Every letter in a text file is stored as a number, and an **encoding** is the table that says which number means which letter. **UTF-8** covers every language and symbol. CSE's titles use curly quotation marks and long dashes - for example DFCC BANK PLC ("BANK") – ... - so the encoding matters. Opened with the wrong one, those characters turn into rubbish symbols.

Why charts?

The share price table has about 284 rows. Reading 284 rows tells you very little. A sorted bar chart shows in one second which companies moved the most. Choosing the right chart matters:

- **Horizontal bars** - company codes are long, and they fit along the side of a chart better than squeezed under each bar.
- **Sorted** - the biggest at one end, so the order means something.
- **Green for up, red for down** - the usual colours for share prices.
- **Never colour alone** - red and green are the hardest pair to tell apart for people with colour blindness, so every bar also says 'up' or 'down' and has a plus or minus sign.
- **Sensible units** - turnover in millions of rupees, not raw numbers with nine digits.

Remember: OCR turns a picture of text back into text, in four steps: get the image, clean it, read it, tidy it. Saving puts the data into CSV, Excel and JSON with the source written inside. Charts make the numbers readable at a glance.

Reading Python

To explain your code, you first need to be able to read it. This chapter explains every piece of Python that appears in your files. Every example is a real line from your own code, so by the end of the chapter you will recognise all of it.

Try it yourself. open `src/ocr_extractor.py` in VS Code or Notepad and keep it beside this document while you read. Find each example in the file as you go.

Comments - notes for people

A line that starts with `#` is a **comment**. Python ignores it completely. It is there for people reading the code. Our files are full of comments explaining why each part exists.

From your code:

```
# Turnover runs into the billions, which is hard to read on an axis,  
# so we show it in millions.
```

From `chart_turnover()`. It explains why the next line divides by a million. Python skips both lines.

Variables - labelled boxes

A **variable** is a name that holds a value. The `=` sign means 'put the thing on the right into the box on the left'. It does not mean 'equals' the way it does in maths.

Think of it like this. a variable is a labelled box. `price = 141.0` means: take a box, write 'price' on the label, and put 141.0 inside.

From your code:

```
millions = data["turnover"] / 1_000_000
```

Take the turnover column, divide every value by a million, and keep the result in a box called `millions`. The underscores in `1_000_000` are only there to make it easier to read - Python ignores them.

Text, and filling in blanks with f-strings

Text in Python is written inside quote marks, like `"hello"`. It is called a **string**. A string that starts with `f` is an **f-string**: anything inside curly brackets `{ }` is replaced with the value of that variable.

Think of it like this. an f-string is a form letter with blanks. `f"Dear {name}"` fills in the blank with whatever name holds.

From your code:

```
image_path = config.IMAGE_DIR / f"{name}_page{page_number}.png"
```

From `pdf_page_to_image()`. The f-string builds a file name like `14364_DE-LISTING OF THE ..._page1.png`. The `/` here is not division - it joins a folder and a file name into one path, like `data/images/14364_..._page1.png`.

Numbers, and True and False

Python has whole numbers like `30`, numbers with a decimal point like `1.5`, and two special values: `True` and `False`. `None` means 'nothing here yet'.

From your code:

```
OCR_TEXT_THRESHOLD = 50
OCR_DPI = 200
```

From `config.py`, and used by your code: a page with fewer than 50 characters is treated as a scan, and pages are turned into images at 200 dpi. Your `tesseract_ready()` also returns `True` or `False` to say whether Tesseract is installed.

Lists - things in order

A **list** holds several values in order, inside square brackets. `.append()` adds one more to the end. `[]` on its own is an empty list, ready to be filled.

Think of it like this. a list is a numbered shopping list. You can add items to the end, and read them back in order.

From your code:

```
rows = []
...
rows.append({"word": word, "Confidence": confidence})
```

From `read_words_with_confidence()`. Start with an empty list, then add one small dictionary for every word Tesseract read.

Dictionaries - values with labels

A **dictionary** holds values under names, inside curly brackets: `{"name": value}`. You get a value back by its name: `reply["ok"]`. The method `.get(name, default)` does the same, but gives back the default instead of crashing if the name is missing.

Think of it like this. a dictionary is a set of labelled drawers. You open a drawer by reading its label, not by counting from the top.

From your code:

```
steps = {"grey": grey_path, "threshold": threshold_path,
         "cleaned": cleaned_path}
```

From `preprocess_image()`. The three image files are kept under labels, so the app can ask for `steps["grey"]` and show the right picture in the right place.

Functions - recipes

A **function** is a named set of steps. `def` starts one. The names in brackets are its **parameters** - the ingredients it needs. `return` hands back the result. The function does nothing until somebody **calls** it by writing its name followed by brackets.

Think of it like this. a function is a recipe. The parameters are the ingredients, the indented lines are the steps, and `return` is the finished dish.

From your code:

```
def pdf_page_to_image(pdf_path, page_number=1, dpi=None):
```

The ingredients are the PDF, which page, and how detailed the image should be. `page_number=1` is a **default** - if nobody says which page, it uses page 1. `dpi=None` means 'not given - use the normal setting'.

if and else - making decisions

`if` runs the indented lines only when a condition is true. `else` runs when it is not. `not` turns true into false and the other way round.

From your code:

```
if page_number < 1 or page_number > len(pdf.pages):  
    return None, f"This PDF has {len(pdf.pages)} page(s)."
```

If someone asks for page 0, or page 5 of a 4-page document, stop and explain instead of crashing. `or` means either one being true is enough. `len(pdf.pages)` is how many pages there are.

for loops - doing the same thing for each item

A **for loop** repeats its indented lines once for every item in a list or table. `enumerate(..., start=1)` also counts them for you: 1, 2, 3.

Think of it like this. a for loop is a teacher going down the register, doing the same thing for each name.

From your code:

```
for i in range(len(data["text"])):  
    word = data["text"][i].strip()  
    confidence = int(data["conf"][i])
```

Tesseract returns separate lists - one of words, one of confidence scores - in the same order. `range(len(...))` counts 0, 1, 2, ... up to the number of words, and `i` picks the matching item from each list.

import - taking a toolbox off the shelf

Python comes with some tools built in. For everything else we **import** a library - a toolbox somebody else wrote. `import pandas as pd` means 'load pandas, and let me call it `pd` for short'. `from X import Y` takes just one tool out of the box.

From your code:

```
import cv2
import numpy as np
import pandas as pd
import pdfplumber
import pytesseract
from PIL import Image
```

cv2 is OpenCV's name in Python. numpy handles grids of numbers - which is what images are. PIL is Pillow, the image library from the lecture. These are the tools from the lecture's OCR pipeline.

try and except - a safety net

Code inside try is attempted. If anything goes wrong, Python jumps to except instead of crashing. We use this wherever the internet or a file might fail.

Think of it like this. try and except is a trapeze artist with a safety net. They attempt the jump; if they fall, the net catches them and the show goes on.

From your code:

```
try:
    version = pytesseract.get_tesseract_version()
    return True, f"Tesseract {version} is installed and ready."
except Exception:
    return False, INSTALL_HELP
```

From `tesseract_ready()`. Ask Tesseract for its version number. If it answers, it is installed. If anything goes wrong, it is not - so return the installation instructions instead of crashing.

with - borrow, use, and give back automatically

with opens something - usually a file - for the indented lines that follow, and closes it again afterwards, even if something goes wrong in the middle.

Think of it like this. borrowing a library book with a promise that it is returned automatically, even if you forget.

From your code:

```
with open(path, "w", encoding="utf-8-sig", newline="") as f:
    f.write(f"# {config.ATTRIBUTION}\n")
```

From `export()`. Open a file for writing ("w"), write into it, and close it automatically at the end. \n means 'start a new line'.

pandas DataFrames - tables in Python

A **DataFrame** is a table with rows and named columns, like a sheet in Excel. pandas is the library that provides it. `pd.DataFrame(rows)` builds one from a list of dictionaries, `df.head(5)` shows the first five rows, and `df["price"]` picks out one column.

From your code:

```
risers = data.nlargest(top, "percentageChange")
fallers = data.nsmallest(top, "percentageChange")
movers = pd.concat([fallers, risers]).sort_values("percentageChange")
```

From `chart_movers()`. Take the 10 biggest rises and the 10 biggest falls, stack the two tables into one with `pd.concat`, and sort it so the chart runs from the biggest fall to the biggest rise.

Giving back two answers at once

A function can return two values separated by a comma, and the caller can catch them in two boxes:

```
return str(image_path), None          # inside the function

image_path, error = pdf_page_to_image(pdf_path, page_number, dpi)
```

Your functions return 'the result, and an error message'. When it works, the error is `None`. When it fails, the result is `None` and the error says why.

List comprehensions - building a list in one line

A **list comprehension** makes a new list from an old one in a single line. From `text_to_lines()`:

```
lines = [line.strip() for line in text.strip().split("\n")]
lines = [line for line in lines if line]
```

The first line reads: 'split the text at every new line, and trim the spaces off each line'. The second: 'keep only the lines that are not empty' - an empty text counts as false.

Formatting numbers

Inside an f-string, a colon after the value sets how a number is shown. Your charts use these:

```
f"{v:+.2f}%"          -> +3.21%   or   -0.50%
f"{v:,.1f}M"          -> 1,234.5M
f"Rs {total / 1_000_000_000:,.2f} bn" -> Rs 0.49 bn
```

+ always shows the sign, , puts commas in thousands, and .2f means two numbers after the decimal point.

if and else in one line

A short choice can be written on one line:

```
direction = ("rose" if change > 0
             else "fell" if change < 0 else "did not move")
```

From `summary()`. Read it as: 'rose if it went up, otherwise fell if it went down, otherwise did not move'.

or - use this, or else that

`dpi = dpi or config.OCR_DPI` means: use the dpi we were given, but if none was given, use the normal setting of 200.

numpy arrays

An image loaded by OpenCV is a **numpy array** - a grid of numbers. Your kernel is a tiny one:

```
kernel = np.ones((1, 1), np.uint8)
```

A grid 1 row by 1 column, filled with ones. `np.uint8` means whole numbers from 0 to 255 - the same kind of number a pixel is.

Named arguments

When a function has many settings, you can name each one as you pass it in, so the order does not matter and the code explains itself:

```
go.Bar(x=movers["percentageChange"], y=movers["symbol"],  
       orientation="h", cliponaxis=False)
```

`orientation="h"` means horizontal bars.

Running a file

Every file in our project can be run on its own to show what it does. You open a terminal in the project folder, switch on the project's Python, and ask Python to run the file:

```
cd D:\DA2009\CSE-Scraping-Project  
.venv\Scripts\activate  
python -m src.ethics
```

`.venv` is the project's own copy of Python with all the libraries installed. `-m src.ethics` means 'run the file `ethics.py` inside the `src` folder'. The part at the bottom of each file that starts `if __name__ == "__main__":` is what runs when you do this.

Your file: src/ocr_extractor.py

Reads scanned PDF pages, where there is no text to extract, using the four-step OCR pipeline from the lecture.

The file has one function for each step, plus one that runs all four in order:

tesseract_ready()	is Tesseract installed?
pdf_page_to_image()	step 1 image acquisition
preprocess_image()	step 2 preprocessing
read_text()	step 3 text recognition
read_words_with_confidence()	step 3 (word by word)
text_to_lines()	step 4 post-processing
ocr_pdf_page()	all four steps together

Before anything - is Tesseract installed?

```
def tesseract_ready():
    try:
        version = pytesseract.get_tesseract_version()
        return True, f"Tesseract {version} is installed and ready."
    except Exception:
        return False, INSTALL_HELP
```

import pytesseract works even when Tesseract is missing - remember, pytesseract is only the telephone. The only sure way to know is to ask Tesseract something. If it cannot give its version number, it is not there, and we show INSTALL_HELP - the installation steps from the lecture, in short - instead of an error.

Step 1 - turning the page into an image

```
def pdf_page_to_image(pdf_path, page_number=1, dpi=None):
    dpi = dpi or config.OCR_DPI
    pdf_path = str(pdf_path)

    with pdfplumber.open(pdf_path) as pdf:
        if page_number < 1 or page_number > len(pdf.pages):
            return None, f"This PDF has {len(pdf.pages)} page(s)."
```

- OCR reads pictures, not PDFs, so the page must become a picture first. pdfplumber's to_image() does that, and gives back a PIL image.
- pdf.pages[page_number - 1] - people count pages from 1, but Python counts from 0. Page 1 is pages[0].
- resolution=dpi - 200 dots per inch, which made a 1,653 x 2,337 pixel image of the de-listing notice.
- The name line takes the file name without its folder or .pdf, so the image is saved as 14364_DE-LISTING ..._page1.png in data/images.

Step 2 - cleaning the image

```
image = cv2.imread(str(image_path))
if image is None:
    return None, "Could not open the image."

# Colour to grey.
grey = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imwrite(grey_path, grey)

# Grey to pure black and white.
threshold = cv2.adaptiveThreshold(
    grey, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    31, 2,
)
cv2.imwrite(threshold_path, threshold)

# The lecture's optional noise removal step. With a 1x1 kernel it
# leaves the image exactly as it was (see the note above).
kernel = np.ones((1, 1), np.uint8)
cleaned = cv2.morphologyEx(threshold, cv2.MORPH_OPEN, kernel)
cv2.imwrite(cleaned_path, cleaned)
```

- `cv2.imread` loads the image as a grid of numbers. If the file cannot be opened, it gives back `None`, so we check for that.
- `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)` - colour to grey. BGR is OpenCV's blue-green-red order.
- `cv2.imwrite` saves each stage as its own picture, so the app can show all of them side by side.

The adaptive threshold has five settings:

Setting	What it means
grey	The picture to work on
255	The value for pixels that pass - white
ADAPTIVE_THRESH_GAUSSIAN_C	Work out each cut-off from the pixels around it, counting nearer pixels more
THRESH_BINARY	Brighter than the cut-off becomes white; darker becomes black
31	The size of the neighbourhood: 31 x 31 pixels. It has to be an odd number, so there is a centre pixel
2	Taken off the local average. A pixel must be a little darker than its surroundings to become black, so slightly grey paper stays white

This is what each stage did to part of the de-listing notice:

DE-LISTING OF THE ORDINARY VOTING SHARES OF ASSOCIATED MOTOR FINANCE COMPANY PLC

In terms of section 14 of the Listing Rules of the Colombo Stock Exchange (CSE), the securities of Associated Motor Finance Company PLC have been delisted from the Official List of the CSE with effect from **6th August 2026**, subsequent to the amalgamation with LB Finance PLC.

The page as it came out of the PDF.

DE-LISTING OF THE ORDINARY VOTING SHARES OF ASSOCIATED MOTOR FINANCE COMPANY PLC

In terms of section 14 of the Listing Rules of the Colombo Stock Exchange (CSE), the securities of Associated Motor Finance Company PLC have been delisted from the Official List of the CSE with effect from **6th August 2026**, subsequent to the amalgamation with LB Finance PLC.

1. Colour removed. Almost the same to the eye - but now one number per pixel instead of three.

DE-LISTING OF THE ORDINARY VOTING SHARES OF ASSOCIATED MOTOR FINANCE COMPANY PLC

In terms of section 14 of the Listing Rules of the Colombo Stock Exchange (CSE), the securities of Associated Motor Finance Company PLC have been delisted from the Official List of the CSE with effect from **6th August 2026**, subsequent to the amalgamation with LB Finance PLC.

2. Black and white. The letters have hard edges and the paper is pure white. Look closely and you can see a few dark specks between words.

DE-LISTING OF THE ORDINARY VOTING SHARES OF ASSOCIATED MOTOR FINANCE COMPANY PLC

In terms of section 14 of the Listing Rules of the Colombo Stock Exchange (CSE), the securities of Associated Motor Finance Company PLC have been delisted from the Official List of the CSE with effect from **6th August 2026**, subsequent to the amalgamation with LB Finance PLC.

3. After the noise step. Identical to picture 2 - 0 pixels changed, because the kernel is 1 x 1.

The specks in picture 2 are exactly the kind of noise that closing with a bigger kernel could remove. They stay, and your code says why: the same change could also rub out full stops and thin parts of letters.

Step 3 - reading the text

```
def read_text(image_array):
    pil_image = Image.fromarray(image_array)
    return pytesseract.image_to_string(pil_image)
```

OpenCV gives us a numpy array. `Image.fromarray()` turns it into a PIL image, the way the lecture passed images to pytesseract. `image_to_string` returns everything Tesseract read as one long piece of text.

```
def read_words_with_confidence(image_array):
    pil_image = Image.fromarray(image_array)
    data = pytesseract.image_to_data(
        pil_image, output_type=pytesseract.Output.DICT)
```

```

rows = []
for i in range(len(data["text"])):
    word = data["text"][i].strip()
    confidence = int(data["conf"][i])
    if word and confidence > 0:
        rows.append({"Word": word, "Confidence": confidence})

return pd.DataFrame(rows)

```

image_to_data gives far more: every word separately, where it is on the page, and a confidence score. Tesseract also lists empty boxes for whole blocks and lines, with a confidence of -1. if word and confidence > 0 keeps only real words.

Step 4 - tidying the result

```

def text_to_lines(text):
    lines = [line.strip() for line in text.strip().split("\n")]
    lines = [line for line in lines if line]

    return pd.DataFrame({
        "Line": range(1, len(lines) + 1),
        "Text": lines,
    })

```

OCR output arrives as one long string full of blank lines. This splits it into lines, trims them, drops the empty ones, and numbers them 1, 2, 3 in a table - which is easy to check, and can be saved as CSV or Excel.

All four steps together - ocr_pdf_page()

```

ready, message = tesseract_ready()
if not ready:
    return {"ok": False, "error": message}

image_path, error = pdf_page_to_image(pdf_path, page_number, dpi) # 1
prepared, steps = preprocess_image(image_path) # 2
text = read_text(prepared) # 3
words = read_words_with_confidence(prepared) # 3
lines = text_to_lines(text) # 4

average_confidence = (round(words["Confidence"].mean(), 1)
                      if not words.empty else 0)

```

(Shortened - the real function checks for an error after every step.) It returns a dictionary with the text, the lines, the word scores, the average confidence and the paths of the three pictures, which is everything the OCR tab shows.

Which page does it read? The app asks Nuhan's PDF reader which pages are scans and reads the **first scanned page**. For the de-listing notice that is page 1, and for the revised-format document it is also page 1 - its pages 2 to 4 have ordinary text.

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.ocr_extractor
```

On a computer without Tesseract, this is the real output:

```
Tesseract is not installed on this computer.
```

```
To install it:
```

1. Download it from <https://github.com/UB-Mannheim/tesseract/wiki>
2. Install it to C:\Program Files\Tesseract-OCR\
3. Tick the option to add it to the system PATH
4. Restart this app

```
The Python part (pip install pytesseract) is already done.
```

Once Tesseract is installed, the same command finds a scanned PDF by itself and prints how many characters it read, the average confidence, the resolution, the first part of the text, and the eight words Tesseract was least sure about. Run it yourself before the viva, so you know what it says for our documents.

Your file: src/storage.py

Saves the data as CSV, Excel and JSON files - and every file carries the credit to the Colombo Stock Exchange inside it.

export() - the time stamps

```
stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
collected = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

`datetime.now()` is the current date and time. `strftime` writes it out in a chosen pattern:

Code	Means	Example
%Y	Year	2026
%m	Month	09
%d	Day	26
%H %M %S	Hours, minutes, seconds	18 27 47

`stamp` goes in the file name - `cse_share_prices_20260926_182747.csv` - so a new save never overwrites an older one. `collected` is the easy-to-read version written inside the file.

The CSV file

```
with open(path, "w", encoding="utf-8-sig", newline="") as f:
    f.write(f"# {config.ATtribution}\n")
    f.write(f"# Collected on: {collected}\n")
    f.write(f"# Rows: {len(df)} Columns: {len(df.columns)}\n")
    df.to_csv(f, index=False)
```

Three lines starting with `#` are written first, then pandas writes the table underneath. This is the top of a real file:

```
# Data source: Colombo Stock Exchange (https://www.cse.lk). Collected
# for academic purposes only - DA 2009, University of Colombo.
# Collected on: 2026-09-26 18:27:47
# Rows: 3 Columns: 3
symbol,price,change
SAMP.N0000,136.75,-0.25
```

(The first line is one long line in the real file.)

- `utf-8-sig` - UTF-8 with a tiny marker at the very start of the file. The marker tells Excel the file is UTF-8, so the curly quotes and dashes in CSE's titles show properly. Without it, Excel can guess wrong and show rubbish characters.
- `newline=""` - lets the CSV writer control the line endings. Without it, Windows can add an extra blank line between every row.
- `index=False` - do not write pandas's row numbers 0, 1, 2 as an extra column.

pandas can read the file back and skip the comment lines:

```
pd.read_csv(path, comment="#")
```

The Excel file - a second sheet

```
source = pd.DataFrame({
    "Item": ["Source", "Website", "Collected on", "Rows",
            "Collected by", "Course"],
    "Value": ["Colombo Stock Exchange", config.BASE_URL, collected,
             len(df), ..., config.COURSE],
})

with pd.ExcelWriter(path, engine="openpyxl") as writer:
    df.to_excel(writer, sheet_name="Data", index=False)
    source.to_excel(writer, sheet_name="Source", index=False)
```

Comment lines would break an Excel sheet, so Excel carries the credit differently: the data goes on a sheet called **Data** and the details on a second sheet called **Source**. openpyxl is the library pandas uses to write .xlsx files. 'Collected by' lists all four of us, with our names.

Be careful: the Excel part is inside try with except Exception: pass, which means 'if it fails, carry on quietly'. The usual reason it fails is that a file of the same name is open in Excel. The CSV and JSON files are still saved, and the app's list of saved files shows exactly which files were made - so a missing Excel file is visible there, even though no error appears.

The JSON file - details above the records

```
contents = {
    "details": {
        "source": config.ATTRIBUTE,
        "collected_on": collected,
        "rows": len(df),
        "collected_by": [config.member_label(m) for m in config.TEAM],
    },
    "records": json.loads(df.to_json(orient="records",
                                     date_format="iso")),
}

with open(path, "w", encoding="utf-8") as f:
    json.dump(contents, f, indent=2)
```

- orient="records" - one { } per row, the same shape CSE sent us.
- date_format="iso" - dates written like 2026-09-25T14:39:57.431, a standard every program understands, instead of big numbers.
- json.loads(...) turns pandas's JSON text back into Python lists and dictionaries, so it can sit inside contents.
- indent=2 - spread over many lines with indents, so a person can read it.

The top of a real JSON file:

```
{
  "details": {
    "source": "Data source: Colombo Stock Exchange ...",
    "collected_on": "2026-09-26 18:27:47",
    "rows": 3,
    "collected_by": [
      "25ada072 - Pasindu",
      "25ada073 - Sasini",
```

```

        "25ada141 - Salaama",
        "24ada076 - Nuhan"
    ]
},
"records": [ ...

```

Remember: three formats, three ways of carrying the credit: comment lines at the top of the CSV, a Source sheet in Excel, and a details section in the JSON. The credit travels inside the file, wherever the file goes.

save_pdf() - safe file names

```

safe = "".join(c for c in filename if c.isalnum() or c in "._-").strip()
safe = safe[:110] or "document"
if not safe.lower().endswith(".pdf"):
    safe += ".pdf"

```

Sasini's crawler names each PDF after its title, but Windows does not allow some characters in file names - such as \ / : * ? " < > |. This keeps only letters, numbers, spaces, dots, underscores and hyphens. `isalnum()` means 'is a letter or a number'.

You can see it in our real files. The title DFCC BANK PLC ("BANK") – BASEL III ... was saved as 14475_DFCC BANK PLC BANK BASEL III ...pdf - the brackets, curly quotes and long dash were removed, which is why there are two spaces in a row. `safe[:110]` keeps names to a sensible length, and or "document" gives a name even if nothing was left.

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.storage
```

Saving a small example in all three formats

```

-----
saved: example_20260926_182747.csv
saved: example_20260926_182747.xlsx
saved: example_20260926_182747.json

```

Reading the CSV back, skipping the comment lines:

symbol	price	change
SAMP.N0000	136.75	-0.25
AEL.N0000	24.50	0.30
ABAN.N0000	1094.25	0.00

It saves a small made-up table of three companies, then reads the CSV back - proving the comment lines do not get in the way. Open the files in data/exports to see the credit for yourself.

Your file: src/analysis.py

Draws the three charts and works out the summary figures, using the cleaned share price table.

The charts are made with **Plotly**, a charting library that makes interactive charts - you can hover over a bar to see its exact value. `import plotly.graph_objects as go` brings it in, and `go.Figure(go.Bar(...))` makes a bar chart.

Colours and style

```
COLOURS = {
    "dark": {
        "up": "#34d399", "down": "#c02626", "bar": "#22d3ee", ...
    },
    "light": {
        "up": "#047857", "down": "#dc2626", "bar": "#0891b2", ...
    },
}
```

The app has a dark and a light theme, so there are two sets of colours. #047857 is a colour written as a **hex code** - three pairs of characters for the amount of red, green and blue. `style()` gives every chart the same plain look: a see-through background so it matches the theme, and faint grid lines so the bars stand out.

`empty_chart()` shows a short message such as 'Collect the market data first' instead of a broken, empty chart.

Chart 1 - the biggest movers

```
data = df[["symbol", "percentageChange"]].dropna()
data = data[data["percentageChange"] != 0]

risers = data.nlargest(top, "percentageChange")
fallers = data.nsmallest(top, "percentageChange")
movers = pd.concat([fallers, risers]).sort_values("percentageChange")

bar_colours = [c["up"] if v > 0 else c["down"]
               for v in movers["percentageChange"]]
labels = [f"{'up' if v > 0 else 'down'} {v:+.2f}%"
          for v in movers["percentageChange"]]
```

1. Keep just the company code and its percentage change, and drop any rows with a gap.
2. Remove companies that did not move - `!= 0` means 'is not zero'.
3. Take the 10 biggest rises and 10 biggest falls, stack them, and sort from the biggest fall to the biggest rise.
4. Colour each bar green or red, and label it with the word and the sign, such as up +3.21% or down -2.10%.

```
figure = go.Figure(go.Bar(
    x=movers["percentageChange"],
    y=movers["symbol"],
    orientation="h",
    ...
```

```

        textposition="outside",
        cliponaxis=False,
    ))

limit = movers["percentageChange"].abs().max() * 1.6
figure.update_xaxes(title_text="Price change (%)",
                    range=[-limit, limit], zeroline=True)

```

- orientation="h" - horizontal bars, with the company codes down the side.
- textposition="outside" puts each label just past the end of its bar. cliponaxis=False stops the longest bar's label being cut off at the edge.
- The axis runs the same distance left and right of zero, and 1.6 times the biggest move, which leaves room for the labels. zeroline=True draws the line at zero that separates the falls from the rises.

Chart 2 - where the money went

```

data = df[["symbol", "turnover"]].dropna()
data = data[data["turnover"] > 0].nlargest(top, "turnover")
data = data.sort_values("turnover")

millions = data["turnover"] / 1_000_000

```

Turnover is the total value of shares traded - how much money changed hands for each company. It has no up or down, only size, so this chart uses **one** colour. A second colour would suggest a difference that does not exist. It shows the top 15, in millions of rupees, labelled like 12.3M.

Chart 3 - the sectors

```

bar_colours = [c["up"] if v > 0 else c["down"] if v < 0
               else c["faint"]
               for v in data["changePercentage"]]

```

A **sector** is a group of companies in the same kind of business - banks, hotels, telecoms. This chart shows how each sector's index moved today. It uses three colours, because a sector can also stay exactly the same: green up, red down, and grey for no change. The chart's height grows with the number of sectors - $\max(400, 26 * \text{len}(\text{data}) + 110)$ - so the bars never get squashed.

The summary figures

```

figures = {"Companies": len(df)}

changes = df["percentageChange"].dropna()
figures["Rose"] = int((changes > 0).sum())
figures["Fell"] = int((changes < 0).sum())
figures["No change"] = int((changes == 0).sum())

total = df["turnover"].dropna().sum()
figures["Turnover"] = f"Rs {total / 1_000_000_000:,.2f} bn"

```

changes > 0 marks every company that rose as True, and .sum() counts the Trues. summary() then turns the figures into sentences, and always ends by saying that one day of data is not investment advice.

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.analysis
```

A real run:

Summary figures

```
-----
Companies      284
Rose           96
Fell           113
No change       75
Turnover        Rs 0.49 bn
```

The All Share Price Index fell to 21,037.35 (-29.54 points). Out of 284 companies traded, 96 rose, 113 fell and 75 finished unchanged, so more companies fell than rose. Total turnover was Rs 0.49 bn. This is one trading day, collected for a university assignment. It is not investment advice.

Building the charts

```
-----
movers         built
turnover        built
sectors         built
```

$96 + 113 + 75 = 284$, so every company is counted exactly once. The **All Share Price Index** (ASPI) is one number that sums up the prices of every company on the exchange - when it falls, the market as a whole went down that day. Your numbers will be different on the day you run it, because the market changes.

Remember: the charts are built from the cleaned table, green and red always come with words and signs, turnover is shown in millions, and the summary checks itself - rose, fell and unchanged add up to the number of companies.

How your part fits into the whole project

Every button in the app follows the same path. The lines marked <-- YOU are yours.

1. You click a button in the app	app.py	
2. The app asks for some data	src/api_client.py	
3. Every request goes through fetch()	src/ethics.py	
a. check robots.txt	src/ethics.py	
b. wait 1.5 seconds	src/ethics.py	
c. send it, saying who we are	src/ethics.py	
d. write it in the request log	src/ethics.py	
4. The reply (JSON) becomes a table	src/api_client.py	
5. Documents are found and downloaded	src/crawler.py	
6. Their text is read	src/pdf_extractor.py	
7. Scanned pages are read by OCR	src/ocr_extractor.py	<-- YOU
8. The table is tidied and checked	src/cleaner.py	
9. Charts are drawn	src/analysis.py	<-- YOU
10. Files are saved (CSV, Excel, JSON)	src/storage.py	<-- YOU

Two more pieces sit beside this path rather than on it: `src/static_scraper.py` shows the ordinary requests-and-BeautifulSoup method, and `src/selenium_scraper.py` shows the real-browser method. The Scrapy spider in `scrapy_project/` does the document search a second way, with the Scrapy framework.

Your part is steps 7, 9 and 10 - the end of the path. At step 7, your OCR reads the scanned pages that Nuhan's PDF reader could get no text from, in the documents Sasini's crawler downloaded. At step 9, your charts are drawn from the table Pasindu's code collected and Sasini's code cleaned. At step 10, your code saves that table as files, with the credit to CSE inside each one. Your part turns what the others collected into something a person can read and keep.

Ethics - what every member must know

The assignment says marks are given for **following ethical scraping practices** and for **being able to explain your work**. Ethics was Nuhan's file, but any of us can be asked about it, so here is the short version.

Ethical and unethical scraping

Ethical (what we do)	Unethical (what we avoid)
Only collect what anyone can see without logging in	Getting past logins, paywalls or CAPTCHAs
Follow the website's robots.txt	Ignoring robots.txt
Wait between requests	Firing requests as fast as possible
Collect no personal information	Collecting data about people
Credit the source	Using the data without saying where it came from

robots.txt

robots.txt is a small file a website publishes to tell programs like ours which parts they may visit. CSE's says:

```
User-agent: *
Disallow:
Disallow: /cgi-bin/
Sitemap: https://www.cse.lk/sitemap.xml
```

User-agent: * means the rules are for every program. The next two lines ask programs to keep out of one folder, /cgi-bin/. None of the addresses we use are in that folder. There is no line asking us to wait between requests.

Think of it like this. robots.txt is a sign on a shop door that says 'Staff only beyond this point'. Nothing physically stops you walking past it - you stay out because you respect it.

What our code does about it

1. Every request goes through one function, `fetch()` in `src/ethics.py`. It checks robots.txt first.
2. It waits **1.5 seconds** between requests, even though CSE does not ask us to.
3. It sends a **User-Agent** with our university, course and email.
4. It writes every request to `logs/request_log.csv`.
5. It saves a copy of each reply so we never ask twice.
6. Every exported file names the **Colombo Stock Exchange** as the source.

Remember: 'We only collect public data, we follow robots.txt, we wait 1.5 seconds between requests, we say who we are, and we credit CSE in every file we save.' If you are asked about ethics, start with that sentence.

Showing your part in the viva

The viva is 8 minutes for the whole team: about 3 minutes of presentation, then 5 minutes of questions about what each member did. This chapter gets you ready for your share of it.

Before the viva

1. Double-click `Run_CSE_App.bat` in the project folder. The app opens in your browser. Keep the black window open - closing it stops the app.
2. Check your tab works, as below.
3. Run your files once from the terminal, so you have seen the output yourself.

Showing it in the app

1. Open the **OCR** tab. Your name is on the label at the top. The line under it says whether Tesseract is installed.
2. The **Scanned document** list only offers documents that need OCR. Choose the de-listing notice (it starts with 14364). Leave **Image detail** at 200 and press **Read it**.
3. Point to the three pictures: colour removed, black and white, and after the noise step - and say honestly that the third one is the same as the second.
4. Point to the text Tesseract read, and the table of least confident words - the ones a person should check.
5. Open the **Data & Export** tab (after collecting the market data on the Market Data tab). Press **Draw the charts**.
6. Under **Save the data**, leave CSV, Excel and JSON ticked and press **Save**. Open the CSV to show the source lines at the top.

Showing it from the terminal

```
tesseract --version           # is Tesseract installed?
python -m src.ocr_extractor    # OCR on a scanned page
python -m src.storage          # save in three formats
python -m src.analysis         # summary figures and charts
```

Your 30-second explanation

Practise saying this out loud until it feels natural. Change the words so they sound like you.

"My part is OCR, saving the files, and the charts. Some CSE announcements are scans - photographs of paper - so there is no text inside them to extract. OCR reads the picture instead, in the four steps from the lecture: I turn the page into an image at 200 dpi, clean it with OpenCV - grey, then adaptive thresholding to pure black and white - read it with pytesseract, and tidy the result into a table with pandas. Tesseract also gives a confidence score, so we show which words to check by hand. Then I save the data as CSV, Excel and JSON, and every file credits the Colombo Stock Exchange inside itself. Finally the charts show the biggest movers, the turnover and the sectors, with words and signs as well as colours."

If the examiner points at your code

These are the parts of your files most likely to be pointed at, and what to say about each:

If they point at...	Say...
<code>pytesseract.get_tesseract_version()</code>	Importing pytesseract works even without Tesseract; asking for its version proves it is really installed.
<code>to_image(resolution=dpi)</code>	Step 1 - the PDF page becomes a 200 dpi image, 1,653 x 2,337 pixels.
<code>cv2.adaptiveThreshold(... 31, 2)</code>	Black or white, with a separate cut-off for each 31 x 31 area, so uneven lighting does not spoil it.
<code>np.ones((1, 1), np.uint8)</code> and <code>MORPH_OPEN</code>	The lecture's optional noise step. A 1 x 1 kernel changes nothing - we measured 0 pixels. A bigger kernel could erase full stops.
<code>image_to_data</code> and <code>confidence > 0</code>	Every word with a confidence score; -1 marks empty boxes, so we skip those.
<code>encoding="utf-8-sig"</code>	So Excel shows CSE's curly quotes and dashes correctly.
<code>sheet_name="Source"</code>	Excel carries the credit on a second sheet, because comment lines would break it.
<code>cliponaxis=False</code>	Stops the longest bar's label being cut off at the edge.

Practice questions

These are the same 50 questions as your section of the team guide: about 30 on your own part, then 20 on the project as a whole. Cover each answer with your hand and try to say it first. By now, every answer should make sense - if one does not, go back to the chapter it comes from.

EASY - THE BASICS

1. Which files did you write?

`src/ocr_extractor.py`, which reads scanned documents; `src/storage.py`, which saves the data as CSV, Excel and JSON; and `src/analysis.py`, which builds the charts and the summary figures.

2. What does OCR stand for, and what does it do?

Optical Character Recognition. It means getting a computer to look at a picture of writing and work out which letters it is seeing, so the words become text you can search and copy.

3. Why did this project need OCR at all?

Because one of the CSE circulars is a scan. Somebody printed the page, signed it and put it through a scanner, so the file contains a photograph of the words. Ordinary PDF extraction returns nothing from it, no matter which library you use.

4. Name some real uses of OCR outside this project.

Digitising invoices, receipts and bank statements. Reading number plates for traffic cameras and car parks. Scanning passports at airports. Reading insurance claim forms. And reading text aloud for people who cannot see well.

5. Which three libraries make up your OCR pipeline?

PIL (Pillow) for opening and handling the image, OpenCV for cleaning the image up, and pytesseract for actually reading the letters. pandas then tidies the result.

6. Which formats do you save the data in?

CSV, Excel and JSON. CSV opens anywhere, Excel is what most people actually use, and JSON keeps the structure for another program to read.

7. What do your charts show?

Three things: the companies whose price moved the most today, the companies with the highest turnover, and how each sector of the market moved.

8. Why do you save the image after each cleaning step?

So they can be shown side by side in the OCR tab - the grey version, the black and white version, and the version after the noise step. It makes the preprocessing visible instead of something that just happens invisibly before the text appears.

9. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

10. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

11. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

12. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

13. What does CSE's robots.txt say?

It blocks one section, /cgi-bin/, and allows everything else. It does not set a Crawl-delay, so it does not ask us to wait between requests at all.

14. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

15. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

16. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

17. What are the four steps of the OCR pipeline?

Image acquisition - turn the PDF page into an image.

Preprocessing - improve the image so the text is easier to read: remove the colour, then turn it pure black and white.

Text recognition - pytesseract reads the letters.

Post-processing - tidy the text into lines with pandas, and save it.

18. How do you turn a PDF page into an image?

With pdfplumber's `to_image()`, at 200 dpi, and then save it as a PNG. OCR works on pictures, not on PDFs, so the page has to become an image before anything else can happen.

```
with pdfplumber.open(pdf_path) as pdf:
    page_image = pdf.pages[page_number - 1].to_image(resolution=dpi)
    page_image.save(str(image_path))
```

19. Why convert the image to grey?

Because colour tells us nothing about which letter something is. A grey image has one brightness value per pixel instead of three colour values, so there is less for the OCR engine to work through and nothing useful has been lost.

20. What is thresholding, and why do you use the adaptive kind?

Thresholding turns every pixel either pure black or pure white, so the letters have hard edges instead of soft grey ones. That is what Tesseract reads best.

We use adaptive thresholding, which works out the cut-off separately for each small part of the page instead of using one value for the whole thing. That matters on a scan, where one side of the page is often darker than the other - a single cut-off would lose the text on the dark side.

```
thresh = cv2.adaptiveThreshold(  
    grey, 255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    31, 2)
```

21. What does the noise removal step do?

In our project, nothing - and it is worth knowing why. It is the optional step from the lecture's OCR example, which uses a 1x1 kernel. A 1x1 kernel leaves every pixel as it was; we checked, and 0 pixels changed.

The idea behind the step is to remove specks left by the scanner, which OCR would otherwise read as stray full stops. But those specks are black dots on a white page, and MORPH_OPEN removes small white spots, so doing it properly would need MORPH_CLOSE with a 2x2 or 3x3 kernel. A bigger kernel can also rub out thin parts of letters, so it would need testing before we changed it. We kept the lecture's setting.

```
kernel = np.ones((1, 1), np.uint8)      # 1x1: changes nothing  
cleaned = cv2.morphologyEx(threshold, cv2.MORPH_OPEN, kernel)
```

22. What does the dpi setting change?

How large the image is. dpi means dots per inch, so a higher number means more detail for the OCR engine to work with, but a bigger image and a slower job.

At 72 dpi small print turns to mush and accuracy collapses. We use 200, which is enough for a printed document. 300 helps with very small print but takes noticeably longer.

23. Why do you convert between OpenCV and PIL?

Because the libraries hold images differently. OpenCV works with numpy arrays - grids of numbers - while PIL has its own image type, and the lecture passed PIL images to pytesseract. `Image.fromarray()` converts the OpenCV result into a PIL image, so we can use OpenCV for the cleaning and pytesseract for the reading.

To be accurate: pytesseract will also take a numpy array directly. We convert anyway so the code follows the lecture's pipeline, where each library gets the kind of image it is normally used with.

24. How do you know whether Tesseract is actually installed?

Importing pytesseract is not enough. It is only a small Python wrapper - the real work is done by the Tesseract program, which on Windows has to be installed separately. The import succeeds even when the program is missing.

So we call `pytesseract.get_tesseract_version()`, which forces it to go and find the program. If that raises an error, we know it is not there and we show the install instructions instead of a crash.

```
def tesseract_ready():  
    try:  
        version = pytesseract.get_tesseract_version()  
        return True, f"Tesseract {version} is installed and ready."  
    except Exception:  
        return False, INSTALL_HELP
```

25. What is the difference between `image_to_string` and `image_to_data`?

`image_to_string` gives you the text as one block. `image_to_data` gives you far more - every word separately, where it sits on the page, and a confidence score from 0 to 100 saying how sure Tesseract is. We use both: the string for reading, and the data so we can show which words are least certain.

26. Why only OCR some pages instead of the whole document?

Because OCR is slow and it guesses, while reading a text layer is instant and exact. The PDF module tells us which pages returned nothing, and we run OCR only on those. Running it over pages we already read perfectly would waste time and introduce mistakes into text that was already correct.

27. How does the source get into each type of file?

Each format needs a different trick.

A CSV gets comment lines at the top, each starting with `#`. Excel shows them, and pandas can skip them when reading the file back with `pd.read_csv(path, comment="#")`.

An Excel file gets a second worksheet called Source, because a spreadsheet has no way to hold a comment.

A JSON file gets a details section above the records.

```
with open(path, "w", encoding="utf-8-sig", newline="") as f:
    f.write(f"# {config.ATTRIBUTE}\n")
    f.write(f"# Collected on: {collected}\n")
    df.to_csv(f, index=False)
```

28. Why does crediting the source matter?

Because a spreadsheet passed on to somebody else with no source attached is how data gets misused. Somebody finds the file six months later and has no idea where the numbers came from or when they were true.

Putting it inside the file means it travels with the data, instead of living in an email that gets lost.

29. What does `utf-8-sig` mean when writing the CSV?

It writes a small marker at the start of the file that tells Excel the file is UTF-8. Without it, Excel on Windows sometimes guesses the wrong encoding and company names with special characters come out as nonsense.

30. Why do you strip odd characters out of the PDF filenames?

Because the announcement titles contain characters Windows will not accept in a filename, such as slashes and quotation marks. We keep only letters, numbers, spaces, dots, underscores and dashes, and cut the name to a sensible length so the full path does not get too long.

31. Why is the movers chart a different shape from the turnover chart?

Because they answer different questions. The movers chart shows direction as well as size, so it spreads out both ways from a zero line, with risers on one side and fallers on the other.

Turnover has no direction - it is only ever an amount - so it is a straightforward bar chart in one colour. Adding a second colour there would suggest a difference that does not exist.

32. Why green and red, and why the extra labels?

Green up and red down is the usual convention for share prices, so it is what anyone reading a market chart expects.

But red and green are the hardest pair to tell apart for people with colour blindness, so every bar also carries the direction in words and a plus or minus sign. The colour is never the only thing telling you which way the price went.

33. Why show turnover in millions rather than the raw number?

Because the raw figures run into the billions of rupees, and an axis labelled 1500000000 is unreadable. Dividing by a million gives numbers a person can compare at a glance, and the axis title says the unit.

34. What is `cliponaxis=False` for?

Without it, Plotly cuts off any label that reaches past the edge of the plot. We put the value at the end of each bar, so the longest bar's label was being clipped. Turning clipping off lets the label draw over the margin, and we also widen the axis range to leave room.

35. Why did `requests` and `BeautifulSoup` not work on the CSE site?

Because the page is dynamic. `requests` downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

36. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

37. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

38. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

39. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

40. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

41. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

42. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

43. Is OCR accurate? Show that you understand its limits.

Not completely, and the output shows it. It runs words together where the spacing is tight, and it misreads logos and stamps as letters because it is looking for letter shapes everywhere.

That is why it is the last resort rather than the first choice. If a document has a real text layer we always read that instead, because it is exact. We also record the confidence score for every word, so anything the engine was unsure about can be checked by hand rather than trusted silently.

44. What are the main challenges in OCR, and how do you deal with them?

Poor image quality - dealt with by scanning at a higher resolution and improving the contrast, which is what our preprocessing does.

Complicated layouts with columns and tables, where the engine has to work out the reading order.

Handwriting, which needs a different kind of model altogether.

Unusual fonts and symbols. And pages that are slightly rotated, which need straightening before anything else, because Tesseract expects roughly horizontal lines.

45. If OCR gave poor results on a document, what would you try?

Raise the dpi first, since small print is the most common cause. Then look at the preprocessed image - if the thresholding has eaten thin strokes or left the background patchy, adjust the block size in the adaptive threshold.

After that, crop to just the part of the page that matters, so the engine is not trying to read a logo or a signature. And if the page is tilted, straighten it, because a few degrees of rotation hurts accuracy more than people expect.

46. What do the summary figures actually tell you about the market?

They give the breadth of the day rather than just the headline. The index can rise while more companies fall than rise, which happens when a few large companies pull the average up.

So we count how many rose, how many fell and how many did not move, alongside the index. If more fell than rose, the day was broadly negative even if the index went up - and the written summary says that in plain words rather than leaving it to the reader.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. robots.txt does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

Word list

Every special word used in this document, from A to Z.

Word	What it means
Adaptive thresholding	Thresholding with a separate cut-off for each small area of the image.
ASPI	All Share Price Index - one number summing up the prices of every company on the exchange.
Closing	Dilation then erosion - fills in small black spots.
Confidence	How sure Tesseract is about a word, from 0 to 100.
CSV	A simple table file: one row per line, commas between columns.
DataFrame	A table in pandas, with rows and named columns.
dpi	Dots per inch - how many pixels are made for each inch of paper.
Dynamic page	A page whose content is added by JavaScript later.
Encoding	The table that says which number stands for which letter, such as UTF-8.
Function	A named set of steps that can be called to do a job.
Greyscale	An image with one brightness number per pixel, no colour.
HTML	The language web pages are written in, using tags.
JavaScript	A language that runs inside the browser and can change the page after it loads.
JSON	A tidy text format for data: names and values.
Kernel	A tiny grid slid over an image to change its shapes.
Library	Code somebody else wrote that we can import and use.
Morphology	Changing the shapes in a black-and-white image, using a kernel.
Noise	Small marks in an image that are not part of the text.
numpy array	A grid of numbers - how OpenCV holds an image.
OCR	Optical Character Recognition - turning a picture of text into text.
OpenCV	A library for working with images, used as cv2 in Python.
Opening	Erosion then dilation - removes small white spots.
PDF	A document file that looks the same on every computer.
PIL / Pillow	A Python library for opening and changing images.
Pixel	One tiny square of a digital picture.
Plotly	The library that draws our interactive charts.
pytesseract	The Python wrapper that sends images to Tesseract.
Request	A message asking a server for something.
Response	The server's reply to a request.

Word	What it means
robots.txt	A file where a website says which parts programs may visit.
Scanned PDF	A PDF that holds a photograph of a page, with no text inside.
Sector	A group of companies in the same kind of business.
Server	A computer that stores a website and answers requests.
Share	A small piece of a company that people can buy and sell.
Static page	A page whose content is already in the HTML file.
Status code	A number on each reply saying how it went - 200 means OK.
Tag	A label in HTML, such as <title> or <table>.
Tesseract	The free OCR program that actually reads the letters.
Thresholding	Turning every pixel pure black or pure white.
Turnover	The total value of shares traded in a day.
URL	The address of a page or file on the web.
User-Agent	The part of a request that says which program sent it.
Variable	A name that holds a value.
Wrapper	Code that lets one program use another, passing messages between them.